

ASSIGNMENT-1

Course: Full Stack Web Development

Course Code: 23CM4121

Assignment No.: 1

Student Name: K SUHAS VARMA

Roll No.: A24126552091

Date: 02-09-2026

1. TITLE

Types of Inheritance in JavaScript Using ES6 Classes and Mixins

2. AIM

To understand and implement four types of inheritance in JavaScript — Single, Multilevel, Hierarchical, and Multiple (via Mixins) — using ES6 class syntax, the extends keyword, and the Object.assign() mixin pattern.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the concept of inheritance and how it promotes code reuse in object-oriented programming.
- To use ES6 class syntax to define base classes and derived classes using the extends keyword.
- To implement Single Inheritance where one derived class (Student) extends one base class (Person).
- To implement Multilevel Inheritance where a chain of three classes — LivingThing, Mammal, Human — is formed.
- To implement Hierarchical Inheritance where two derived classes (Circle and Rectangle) both extend one base class (Shape).
- To simulate Multiple Inheritance in JavaScript using the Object.assign() mixin pattern, combining Manager and Developer behaviour into the Employee class's prototype.
- To run all four inheritance patterns from a single file (Assignment_1.js) and verify the console output against expected results.

4. THEORY

Inheritance — a core principle of object-oriented programming that allows a class (derived/child) to acquire properties and methods of another class (base/parent), enabling code reuse and a logical hierarchy.

ES6 Class Syntax — JavaScript ES6 introduced the class keyword as syntactic sugar over prototype-based inheritance. Methods defined in a class body are added to the class's prototype and become available to every instance, and to any subclass that extends it.

extends Keyword — used in a class declaration to set up the prototype chain between a derived class and its base class, giving the derived class access to all non-private members of the base class.

Method Inheritance — a subclass automatically gains access to every method defined on its parent class — and on that parent's own parent, and so on up the chain — without redefining them, simply by using extends.

Single Inheritance — one derived class (Student) inherits from one base class (Person). The simplest form: Student gains the inherited walk() method and adds its own study() method.

Multilevel Inheritance — a chain of three classes: LivingThing is the root, Mammal extends LivingThing, and Human extends Mammal. Each level adds its own method — breathe(), giveBirth(), speak() — and a Human instance has access to all three.

Hierarchical Inheritance — two or more derived classes (Circle, Rectangle) independently extend the same base class (Shape). Each child class adds its own unique method — area() or perimeter() — while sharing Shape's draw() method.

Multiple Inheritance (Mixin) — JavaScript does not support multiple extends. Object.assign(Employee.prototype, Manager, Developer) copies the manage() and code() methods from two plain mixin objects directly onto Employee's prototype, so an Employee instance ends up with work(), manage(), and code() all available together.

5. ALGORITHM / PROCEDURE

Create a single JavaScript file named Assignment_1.js containing all four inheritance demonstrations.

Single Inheritance: define the base class Person with a walk() method; define Student extends Person with its own study() method; create a Student object and call both walk() and study().

Multilevel Inheritance: define LivingThing with breathe(); define Mammal extends LivingThing with giveBirth(); define Human extends Mammal with speak(); create a Human object and call all three methods.

Hierarchical Inheritance: define the base class Shape with draw(); define Circle extends Shape with area() and Rectangle extends Shape with perimeter(); create one Circle and one Rectangle object and call draw() plus each one's own method.

Multiple Inheritance (Mixins): define the base class Employee with work(); define two plain objects, Manager (with manage()) and Developer (with code()); use Object.assign(Employee.prototype, Manager, Developer) to merge both mixins into Employee's prototype; create an Employee object and call work(), manage(), and code().

Run the file using Node.js (node Assignment_1.js) and verify the console output against expected results. Record expected vs. actual output for each test case.

6. PROGRAM

Program: Assignment_1.js

// 1. SINGLE INHERITANCE

```
class Person {  
  walk() {  
    console.log("Person can walk");  
  }  
}
```

```
class Student extends Person {  
  study() {  
    console.log("Student studies");  
  }  
}
```

```
console.log("1. Single Inheritance:");  
let student1 = new Student();  
student1.walk();  
student1.study();
```

// 2. MULTILEVEL INHERITANCE

```
class LivingThing {  
  breathe() {  
    console.log("Living thing breathes");  
  }  
}
```

```
class Mammal extends LivingThing {  
  giveBirth() {  
    console.log("Mammal gives birth");  
  }  
}
```

```
class Human extends Mammal {
```

```
    speak() {  
        console.log("Human can speak");  
    }  
}
```

```
console.log("\n2. Multilevel Inheritance:");  
let human = new Human();  
human.breathe();  
human.giveBirth();  
human.speak();
```

// 3. HIERARCHICAL INHERITANCE

```
class Shape {  
    draw() {  
        console.log("Shape can be drawn");  
    }  
}
```

```
class Circle extends Shape {  
    area() {  
        console.log("Circle has an area");  
    }  
}
```

```
class Rectangle extends Shape {  
    perimeter() {  
        console.log("Rectangle has a perimeter");  
    }  
}
```

```
console.log("\n3. Hierarchical Inheritance:");
```

```
let circle = new Circle();  
circle.draw();
```

```
circle.area();
```

```
let rectangle = new Rectangle();
```

```
rectangle.draw();
```

```
rectangle.perimeter();
```

// 4. MULTIPLE INHERITANCE USING MIXINS

```
class Employee {
```

```
  work() {
```

```
    console.log("Employee is working");
```

```
  }
```

```
}
```

```
const Manager = {
```

```
  manage() {
```

```
    console.log("Employee is managing the team");
```

```
  }
```

```
};
```

```
const Developer = {
```

```
  code() {
```

```
    console.log("Employee is writing code");
```

```
  }
```

```
};
```

```
Object.assign(Employee.prototype, Manager, Developer);
```

```
console.log("\n4. Multiple Inheritance using Mixins:");
```

```
let employee = new Employee();
```

```
employee.work();
```

```
employee.manage();
```

```
employee.code();
```

7. CODE EXPLANATION

Code	Explanation
<code>class Person { ... }</code>	Defines the base class Person with a <code>walk()</code> method that logs a message to the console.
<code>class Student extends Person</code>	Declares Student as a derived class that inherits the <code>walk()</code> method from Person using the <code>extends</code> keyword, and adds its own <code>study()</code> method.
<code>class LivingThing / class Mammal extends LivingThing / class Human extends Mammal</code>	Forms a three-level inheritance chain: Human inherits <code>breathe()</code> from LivingThing and <code>giveBirth()</code> from Mammal, while adding its own <code>speak()</code> method.
<code>class Shape { draw() { ... } }</code>	Defines the base class Shape with a <code>draw()</code> method that is shared by every shape class that extends it.
<code>class Circle extends Shape / class Rectangle extends Shape</code>	Hierarchical inheritance: Circle and Rectangle independently extend Shape, each adding its own method — <code>area()</code> and <code>perimeter()</code> respectively — while both share <code>draw()</code> .
<code>const Manager = { manage() {...} }; const Developer = { code() {...} };</code>	Plain JavaScript objects (mixins), each defining one method that will be shared with another class.
<code>Object.assign(Employee.prototype, Manager, Developer)</code>	Copies the <code>manage()</code> and <code>code()</code> methods from the Manager and Developer mixin objects directly onto Employee's prototype, simulating multiple inheritance since Employee now has methods from more than one source.
<code>new Student() / new Human() / new Circle() / new Rectangle() / new Employee()</code>	Creates instances of each derived class. Every object has access to its own methods plus every method inherited along its prototype chain.
<code>student1.walk() / human.speak() / circle.area() / employee.manage() etc.</code>	Method calls that demonstrate both inherited and own methods work correctly on each object.

8. TEST CASES AND EXECUTION DATA

TC-01	Run Assignment_1.js — Single Inheritance: <code>student1.walk(),</code> <code>student1.study()</code>	Prints "Person can walk" then "Student studies"	Both lines printed correctly	Pass
TC-02	Run Assignment_1.js — Multilevel Inheritance: <code>human.breathe(),</code> <code>human.giveBirth(),</code>	Prints "Living thing breathes", "Mammal gives birth", "Human can speak"	All three lines printed in correct order	Pass

TC-01	Run Assignment_1.js — Single Inheritance: student1.walk(), student1.study() human.speak()	Prints "Person can walk" then "Student studies"	Both lines printed correctly	Pass
TC-03	Run Assignment_1.js — Hierarchical Inheritance: circle.draw(), circle.area(), rectangle.draw(), rectangle.perimeter()	Prints "Shape can be drawn"/"Circle has an area" then "Shape can be drawn"/"Rectangle has a perimeter"	All four lines printed correctly for both objects	Pass
TC-04	Run Assignment_1.js — Multiple Inheritance (Mixins): employee.work(), employee.manage(), employee.code()	Prints "Employee is working", "Employee is managing the team", "Employee is writing code"	All three lines printed correctly	Pass

9. OUTPUT

Output: Running Assignment_1.js

Running "node Assignment_1.js" in the terminal executes all four inheritance demonstrations in sequence and prints the console output shown below, confirming that Single, Multilevel, Hierarchical, and Multiple (Mixin) inheritance all work as expected.

```
PS C:\Users\sures\OneDrive\Documents\FS_LAB\Assignments\Assignment_1> node Assignment_1.js
1. Single Inheritance:
Person can walk
Student studies

2. Multilevel Inheritance:
Living thing breathes
Mammal gives birth
Human can speak

3. Hierarchical Inheritance:
Shape can be drawn
Circle has an area
Shape can be drawn
Rectangle has a perimeter

4. Multiple Inheritance using Mixins:
Employee is working
Employee is managing the team
Employee is writing code
PS C:\Users\sures\OneDrive\Documents\FS_LAB\Assignments\Assignment_1> |
```

10. RESULT

Thus, four types of inheritance — Single, Multilevel, Hierarchical, and Multiple (via Mixins) — were successfully implemented in JavaScript using ES6 class syntax and the `Object.assign()` mixin pattern. Each pattern was verified by running `Assignment_1.js` in Node.js, with all four test cases passing and the console output matching the expected results.